

The Visitor Pattern

A lambda-calculus interpreter in Python

CS413

Fall 2026

Why use a visitor?

An expression tree supports several operations:

- Evaluate the expression to obtain a value.
- Compute its free variables.
- Other possible operations include printing or type checking.

Each operation needs a case for every kind of expression.

Visitor groups the cases for one operation in one object. The expression classes store syntax and provide an accept method.

The roles in the Visitor Pattern

Role	In this interpreter
Element	An expression node, such as <code>DOEint</code>
<code>accept(visitor)</code>	Calls the visitor method for that node's form
Visitor interface	<code>DOExpVisitor[R]</code> declares one method per form
Concrete visitor	<code>EvaluateVisitor</code> or <code>FreeVariableVisitor</code>
Client	Calls <code>expr.accept(visitor)</code> and uses the result

A node delegates to the visitor

```
@dataclass
class D0Eint(D0E000):
    arg1: sint

    def accept[R](self, visitor: D0ExpVisitor[R]) -> R:
        return visitor.visit_int(self)
```

- The node knows that it is an integer expression.
- It passes itself to `visit_int`, so the visitor can read `arg1`.
- The supplied visitor determines what to do with that integer.

One node, two operations

```
# In EvaluateVisitor:
def visit_int(self, dexp: D0Eint) -> d0val:
    return D0Vint(dexp.arg1)

# In FreeVariableVisitor:
def visit_int(self, dexp: D0Eint) -> frozenset[dvar]:
    return frozenset()
```

```
expr = D0Eint(7)
expr.accept(EvaluateVisitor())      # D0Vint(7)
expr.accept(FreeVariableVisitor())  # frozenset()
```

The expression stays the same. The visitor selects the operation.

How dispatch works

For `expr = D0Eint(7)` and `v = EvaluateVisitor()`:

1. The client calls `expr.accept(v)`.
2. Python finds `accept` on `D0Eint`.
3. That method calls `v.visit_int(expr)`.
4. Python finds `visit_int` on `EvaluateVisitor`.
5. The visitor returns `D0Vint(7)` through `accept`.

The behavior depends on both the node class and the visitor class. This is the Visitor Pattern's **double dispatch**.

Here, explicit method names such as `visit_int` encode the node case.

The visitor controls recursive traversal

```
# In FreeVariableVisitor:  
def visit_op2(self, dexp: D0Eop2) -> frozenset[dvar]:  
    return dexp.arg1.accept(self) | dexp.arg2.accept(self)
```

For D0Eop2("+", D0Evar("x"), D0Eint(1)):

1. The root dispatches to visit_op2.
2. The visitor visits the left child, obtaining {"x"}.
3. It visits the right child, obtaining the empty set.
4. It returns their union, frozenset({"x"}).

accept dispatches one node. Visitor methods decide which children to visit.

The extension tradeoff

Adding an operation

Create a new visitor and implement its cases. Existing expression classes and existing visitors can stay unchanged.

Adding an expression form

Add the node class and its accept method, extend the visitor interface, and add the new case to every concrete visitor.

Visitor is useful when the node forms are relatively stable and new operations are common.

The visitor result type

```
class D0ExpVisitor[R] (ABC):  
  @abstractmethod  
  def visit_int(self, dexp: D0Eint) -> R:  
    raise NotImplementedError  
  # One abstract method for each expression form.
```

- EvaluateVisitor: R = d0val.
- FreeVariableVisitor: R = frozenset[dvar].
- Each accept[R] has a local type parameter linking the visitor's result type to its own return type.

This source requires Python 3.12 or later. Annotations support static checking. Abstract methods require concrete visitors to implement every case.

The interpreter's public operations

```
def d0exp_fvset(dexp: d0exp) -> frozenset[dvar]:  
    return dexp.accept(FreeVariableVisitor())  
  
def d0exp_evaluate(dexp: d0exp,  
                  denv: d0env = ENVnil()) -> d0val:  
    return dexp.accept(EvaluateVisitor(denv))
```

Expressions describe programs. Values describe results. Environments bind names to values.

The interpreter builds syntax with constructors. It does not include a parser.

Expression constructors

- Constants and operators: D0Eint, D0Ebtf, D0Eop1, D0Eop2.
- Variables and functions: D0Evar, D0Elam, D0Efix, D0Eapp.
- Binding and control: D0Elet, D0Eif0.
- Pairs: D0Epair, D0Epfst, D0Epsnd.

```
D0Eop2("+", D0Eint(2), D0Eint(3))  # syntax for 2 + 3
D0Elam("x", D0Evar("x"))            # lambda x. x
D0Efix("f", "x", body)              # fix f(x). body
```

The meaning of `arg1`, `arg2`, and `arg3` depends on the constructor.

Free variables and lexical binding

A variable occurrence is free when no enclosing binder binds it.

$$\text{FV}(x) = \{x\}$$

$$\text{FV}(42) = \emptyset$$

$$\text{FV}(\lambda x. e) = \text{FV}(e) \setminus \{x\}$$

$$\text{FV}(\text{fix } f(x). e) = \text{FV}(e) \setminus \{f, x\}$$

$$\text{FV}(\text{let } x = e_1 \text{ in } e_2) = \text{FV}(e_1) \cup (\text{FV}(e_2) \setminus \{x\})$$

Operators, application, and pairs combine their children's free names. A conditional includes names from its condition and both branches.

A let binder covers only its body

`let x = x + 1 in x + y`

- Initializer: free names are $\{x\}$.
- Body: free names are $\{x, y\}$ before accounting for the binder.
- Remove the bound x from the body's set, then take the union.

$$FV = \{x\} \cup (\{x, y\} \setminus \{x\}) = \{x, y\}$$

```
def visit_let(self, dexp):  
    return (dexp.arg2.accept(self)  
            | (dexp.arg3.accept(self) - {dexp.arg1}))
```

Environments and shadowing

ENVnil() is empty. ENVcns(name, value, tail) adds a binding. Lookup searches the newest binding first.

```
old = ENVcns("x", DOVint(1), ENVnil())  
new = ENVcns("x", DOVint(2), old)  
  
d0env_search(new, "x")  # DOVint(2)  
d0env_search(old, "x")  # DOVint(1)
```

Frozen environment nodes preserve earlier scopes when a new node extends them.

An unbound name returns DOV000(), the implementation's error sentinel.

Evaluating let introduces a new scope

```
def visit_let(self, dexp: D0Elet) -> d0val:  
    value = dexp.arg2.accept(self)  
    denv = ENVcns(dexp.arg1, value, self.denv)  
    return dexp.arg3.accept(EvaluateVisitor(denv))
```

1. Evaluate the initializer in the current environment.
2. Extend that environment with the new binding.
3. Evaluate the body using a new visitor for the extended scope.

The initializer cannot use the new binding. The caller's scope stays intact.

Values and closures

D0Vint and D0Vbtf hold scalar results. D0Vpair holds two evaluated values.

A **closure** stores a function expression together with its defining environment.

```
def visit_lam(self, dexp):  
    return D0Vlam(self.denv, dexp)  
  
def visit_fix(self, dexp):  
    return D0Vfix(self.denv, dexp)
```

The body runs when the function is applied. This implementation captures the whole environment, without trimming it using free-variable analysis.

Function application uses call by value

```
dfun = dexp.arg1.accept(self) # function in caller's scope
darg = dexp.arg2.accept(self) # argument in caller's scope

# When dfun is a DOVlam:
dlam = dfun.arg2
denv = ENVcns(dlam.arg1, darg, dfun.arg1)
return dlam.arg2.accept(EvaluateVisitor(denv))
```

- Evaluate the function, then the argument, even if the body ignores it.
- Bind the parameter in the *closure's* environment.
- Evaluate the body in that extended environment.

Lexical scoping preserves the defining environment

Surface notation for a constructor-built expression:

```
let x = 10 in  
let f = (lam y. x + y) in  
let x = 100 in  
f(1)
```

1. Defining `f` captures the environment where `x = 10`.
2. Calling `f(1)` binds `y = 1` in that saved environment.
3. The body finds `x = 10` and `y = 1`.

Result: `D0Vint(11)`. The caller's `x = 100` does not replace the captured binding.

Recursive closures bind their own name

For fix `fact(n). if n == 0 then 1 else n * fact(n - 1)`, applying the resulting closure to 3 produces `DOVint(6)`.

```
# When dfun is a DOVfix:
dfix = dfun.arg2
denv = ENVcns(dfix.arg1, dfun, dfun.arg1) # function name
denv = ENVcns(dfix.arg2, darg, denv)      # parameter
return dfix.arg3.accept(EvaluateVisitor(denv))
```

Each call extends the defining environment with the same recursive closure and the current argument. Closure creation needs no cyclic environment.

Conditionals choose which branch to visit

```
def visit_if0(self, dexp: D0Eif0) -> d0val:
    cond = dexp.arg1.accept(self)
    if not isinstance(cond, D0Vbtf):
        raise TypeError(f"D0Vbtf(...) expected: {cond}")
    return (dexp.arg2 if cond.arg1 else dexp.arg3).accept(self)
```

Despite its name, if0 requires a Boolean value in this interpreter.

```
term = D0Eif0(D0Ebtf(True), D0Eint(7),
              D0Eop2("/", D0Eint(1), D0Eint(0)))
d0exp_evaluate(term)  # D0Vint(7)
```

The unselected branch never evaluates.

Pairs, operators, and runtime checks

- Pairs evaluate both components, left to right. Projections require `D0Vpair`.
- Unary operators `+1` and `-1` increment and decrement integers.
- Binary arithmetic supports `+`, `-`, `*`, and `/`.
- Comparisons `<`, `>`, `<=`, `>=`, `==`, and `!=` take integers and return `D0Vbtf`.

```
d0exp_evaluate(D0Eop2("/", D0Eint(-3), D0Eint(2)))  
# D0Vint(-2): division uses Python's floor division (//)
```

Wrong operand types raise `TypeError`. Division by zero raises `ZeroDivisionError`. There is no static type checker here.

Analysis and evaluation visit different children

```
term = D0Eif0(D0EbtF(True),  
              D0Eint(7), D0Evar("missing"))  
  
d0exp_fvset(term)      # frozenset({"missing"})  
d0exp_evaluate(term)   # D0Vint(7)
```

Free-variable analysis visits all branches because it describes syntax.

Evaluation visits the selected branch because it follows execution.

Both use the same accept protocol. Each visitor defines its own traversal.

Questions for discussion

1. What is $FV(\lambda x. x + y)$?
2. What happens when we evaluate $(\lambda x. 42)(1/0)$?
3. Why does the lexical-scope example return 11?
4. Which classes change if we add a `DOEneg` expression?
5. Does `accept` itself recursively visit every child?

Discussion answers

1. $\{y\}$, because the lambda binds x .
2. Argument evaluation raises `ZeroDivisionError` before the body runs.
3. The closure captures $x = 10$, and application adds $y = 1$.
4. Add `D0Eneg.accept`, declare `visit_neg` in `D0ExpVisitor`, and implement it in each concrete visitor.
5. No. Each visitor method decides how to visit children.

Reading the implementation

Source: `../lambda1_vp.py`

1. Expression dataclasses and their `accept` methods.
2. `D0ExpVisitor[R]` and its abstract cases.
3. `FreeVariableVisitor` and the `d0exp_fvset` wrapper.
4. Runtime values, environments, and `d0env_search`.
5. `EvaluateVisitor` and the `d0exp_evaluate` wrapper.

The node supplies its expression form. The visitor supplies the operation.